EduMatrix Backend System Manual

Exhaustive Architectural Specification, Database Models & API Engine Reference

FastAPI (Python 3.10) | SQLAlchemy ORM | dlib Biometrics | WebSockets | Firebase & SMS Gateway

SYSTEM OVERVIEW

EduMatrix Backend is a high-throughput, asynchronous ASGI system engineered in Python 3.10 and FastAPI. It orchestrates sub-millisecond AI facial biometrics, relational academic data models, real-time GIET ERP ledger synchronization for B.Tech (2023-2027), multi-channel parent communication pipelines, and full-duplex WebSocket messaging.

1. Technology Stack & Runtime Infrastructure

Technical stack components and runtime configurations

Subsystem	Technology	Technical Specifications & Role
Language & Runtime	Python 3.10 (x64)	Asynchronous execution, strict type-hints, Python bytecode compilation
Application Server	FastAPI (ASGI)	Async request handlers, Pydantic data validation, OpenAPI 3.0 auto-specs
ASGI Web Server	Uvicorn (StatReload)	ASGI server worker running on 0.0.0.0:8000 with thread pooling
ORM / Persistence	SQLAlchemy 2.0	Declarative schema mappings, foreign key cascades, connection pooling
Primary Database	MySQL & SQLite3	Production/development database engines for high-integrity academic data
AI Biometric Engine	dlib & face_recognition	ResNet-34 deep metric network generating 128-d floating point embeddings
Authentication	JWT & Bcrypt (passlib)	Stateless bearer token encryption with HS256 algorithm and salted hashes
Real-Time Engine	WebSockets (FastAPI)	Duplex WebSocket manager for group chats, live statuses and direct messaging
Notification Layer	Firebase Admin & SMS	FCM multicast push notifications and automated parent SMS dispatch

2. Database Architecture & Data Models (SQLAlchemy)

Relational entities, tables, and relationships across the schema

- **User, Role & Permission (RBAC):** Models 'users', 'roles', 'permissions', and 'role_permissions'. Provides strict authorization separation across Admin, Teacher, and Student roles with dynamic HOD/Class Teacher elevation.
- **Student Entity ('students'):** Stores user_id, roll_no (e.g. 23CSE706), program (B.Tech), department (CSE), year (4), semester (7), section (G), guardian_name, and guardian_mobile.
- **Teacher Entity ('teachers'):** Stores user_id, employee_id, department, designation, qualifications, and assigned class teacher sections.
- **Biometric Attendance ('attendance', 'attendance_sessions'):** Persists student_id, subject_id, date, period, and verification status (Present, Late, Absent, Excused) with verification timestamps.
- **Smart Fee Ledger ('student_fees', 'fee_payment_transactions'):** Tracks breakdown of dues (tuition_fee_due, hostel_fee_due, exam_fee_due, library_fine_due) and transaction logs with digital receipts.
- **Parent Request Entity ('parent_payment_requests'):** Stores parent payment requests, parent phone number, amount, fee category, status (PENDING/PAID), and creation timestamp.
- **Marks & Assessments ('marks', 'subject_assessments'):** Records Midterm 1, Midterm 2, assignments, and lab scores for real-time SGPA and CGPA projections.
- **Communication & Notices ('notices', 'notice_reads', 'notifications'):** Stores institutional broadcasts, read receipts, and student notification histories.

3. AI Facial Recognition Engine Architecture

Deep Metric Biometric pipeline implementation (dlib / OpenCV / ResNet-34)

*** Lifespan In-Memory Encoding & Distance Matching Pipeline**

1. Startup Lifespan Encodings (app/face_recognition/recognize.py):
   - On server boot, load_known_faces() traverses images/students/ and images/encodings/.
   - Extracts 128-dimensional floating point vectors via dlib ResNet-34 model and caches them in known_face_encodings.

2. Multi-Target Frame Capture & Detection:
   - Client sends multipart camera frames to POST /attendance/verify or /attendance/take.
   - face_recognition.face_locations() runs HOG/CNN detector to locate all face bounding boxes.

3. Euclidean Distance Calculation:
   - face_recognition.face_encodings() computes candidate vectors.
   - face_recognition.face_distance() computes distance against known student encodings.
   - Strict threshold rule: distance <= 0.50 triggers positive match with anti-spoofing validation.

4. Database Registration:
   - Verifies student belongs to the scheduled section & subject -> saves record into Attendance table.

4. GIET ERP Academic Ledger & Data Normalization

Live database synchronization and batch normalization for B.Tech (2023 - 2027)

*** Ledger Calculation & Dual-Layer Synchronization (app/routes/erp_routes.py)**

1. Batch Normalization:
   - Standardized across all endpoints to B.Tech cohort '2023 - 2027'.

2. Dynamic Attendance Aggregation:
   - Computes: Overall Attendance % = (Total Classes Attended / Total Classes Conducted) * 100.
   - Filters authorized timetable subjects for student's department, semester, and section.

3. Live Synchronization:
   - Synchronizes mobile Student Dashboard and GIET ERP Screen to guarantee identical real-time numbers.

4. Marks & CGPA Computation:
   - Aggregates internal examination scores (Midterm 1, Midterm 2, Assignment, Lab) to compute accurate SGPA/CGPA.

5. Smart Fee Portal & Automated Communication Pipeline

Fee management, payment simulation, and multi-channel SMS/WhatsApp notification dispatch

*** End-to-End Parent Notification Workflow (app/routes/fee_routes.py)**

1. Fee Dues Query (GET /fees/dues):
   - Returns real-time dues for Tuition, Hostel, Exam, and Library alongside registered parent details.

2. Payment Processing (POST /fees/pay):
   - Processes payments via UPI/Card, decrements due balance, and returns digital receipt with transaction ID.

3. Parent Request Dispatch (POST /fees/request-parent):
   - Persists request into ParentPaymentRequest and creates in-app Notification record.
   - Dispatches Firebase Cloud Messaging (FCM) push notification to student and linked accounts.
   - Dispatches automated SMS/WhatsApp gateway alert to the parent phone number:
     'Dear [Parent], GIET University has generated a fee payment request for [Student] ([Roll No]) of Rs. [Amount] for [Category] Fee. Please pay via EduMatrix portal.'
   - Built with strongly typed DTOs (FeePayRequestDto, ParentPaymentRequestDto) and @JvmSuppressWildcards.

6. AI Placement Readiness & WebSocket Chat Subsystems

Generative AI career suite and full-duplex WebSocket architecture

A. AI Placement & Career Readiness (app/routes/placement_routes.py)

- **AI Mock Interview Engine:** Uses Google Gemini LLM to generate dynamic branch-tailored technical/HR interview questions. Analyzes spoken/text candidate responses with scored feedback.
- **ATS Resume CV Builder:** Generates standardized single-page industry resumes with PDF rendering.
- **Mock Test Center:** Aptitude and coding assessments with automatic grading and percentile calculation.

B. Real-Time WebSockets Engine (app/utils/websocket_manager.py)

- **Connection Pool:** Maintains active WebSocket connections dictionary keyed by user_id.
- **Group & Direct Chats:** Broadcasts messages to entire class sections or routes encrypted direct messages between student and faculty.

7. Complete Backend Router Map (36 Route Modules)

Inventory of registered FastAPI routers in app/routes/

Router Module	URL Prefix	Core Functionality
auth_routes.py	/auth	JWT Login, Token Refresh, Password Reset, Profile, HOD detection
attendance_routes.py	/attendance	Facial recognition verification, live class scanning, manual edits
student_routes.py	/student	Student profile, dynamic attendance history, timetable, leaves
teacher_routes.py	/teacher	Teacher dashboard, student lookup, availability toggle, class lists
admin_routes.py	/admin	Master institutional management, system statistics, analytics
erp_routes.py	/erp	GIET ERP Academic Ledger endpoints, live attendance & CGPA
fee_routes.py	/fees	Smart Fee portal dues, payment simulation, parent SMS dispatch
notification_routes.py	/notifications	Institutional circulars, notice board, FCM push broadcasts
placement_routes.py	/placement	AI Mock interviews, ATS resume generation, readiness scoring
quiz_routes.py	/quiz	Standardized mock tests, daily trivia, gamified rewards store
chat_routes.py	/chat	WebSocket endpoints, group messages, 1-on-1 chat history
spaces_routes.py	/spaces	Smart library seat allocation, study room reservations
canteen_routes.py	/canteen	EduMatrix Bites online food ordering and live queue status
pulse_routes.py	/pulse	Campus mood tracking and mental health wellness metrics
quest_routes.py	/quests	Gamified student productivity quests and reward redemption

8. Deployment & Execution Instructions

Terminal commands and server startup guidelines

*** Executing FastAPI Backend Server**

1. Open Terminal or PowerShell:
   cd "D:\7th Sem\Project\EduMatrix"

2. Run Backend with Python 3.10:
   py -3.10 -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

3. Live URLs:
   - Web Console: http://localhost:8000/
   - OpenAPI Interactive Docs: http://localhost:8000/docs
   - Alternative ReDoc: http://localhost:8000/redoc
   - Network Ping: http://10.49.123.225:8000/ping